

LexAegis AI — Beginner Deployment Guide

For someone who has never deployed a full-stack application before. Follow it top to bottom. By the end you will have LexAegis running on the public internet: the frontend on **Vercel**, the backend + vector database on **Railway**, using **Gemini** for the LLM and **Supabase** for login.

You do **not** need Docker installed to deploy to Vercel/Railway — they build the images for you in the cloud. (Docker is only for running everything on your own machine; see [DOCKER_FOR_BEGINNERS.md](#).)

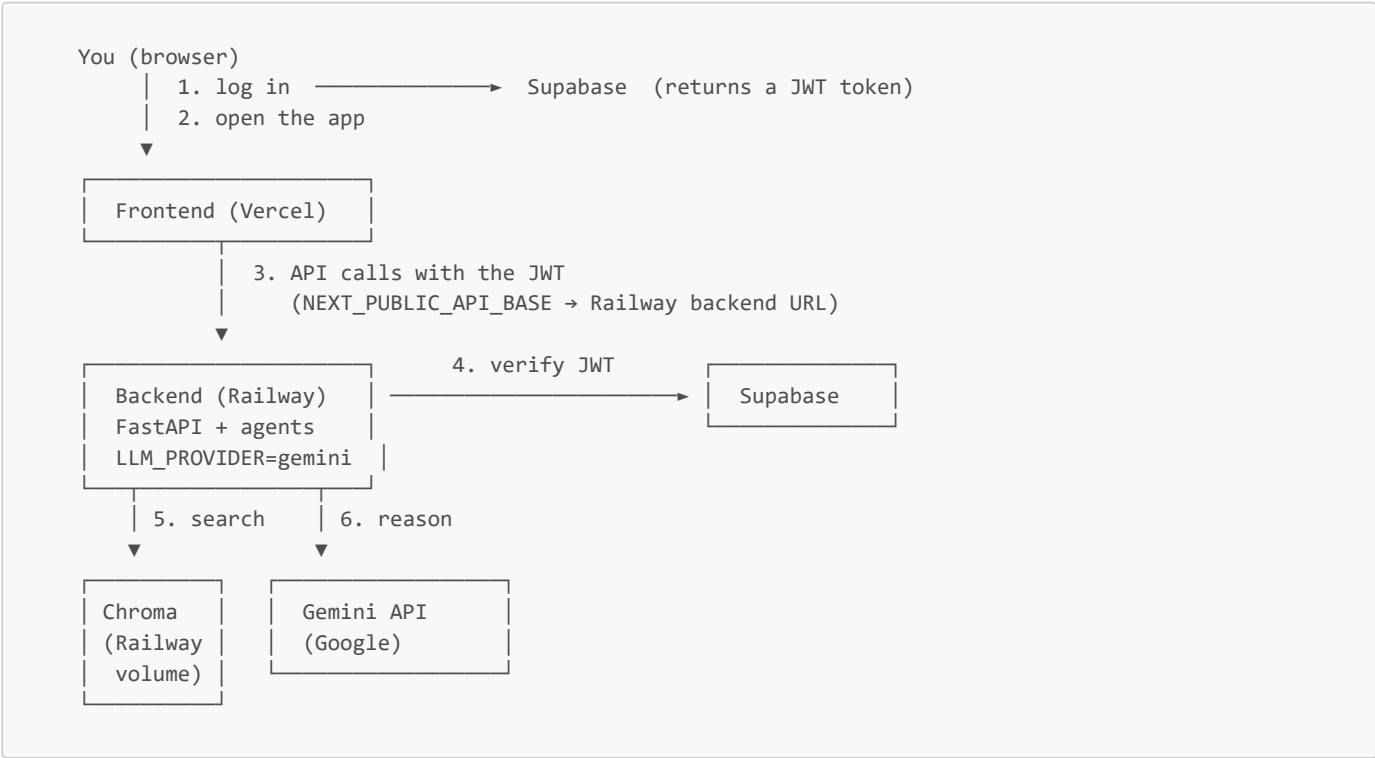
Time required: ~60–90 minutes the first time. Cost: \$0 to start — all five services have free tiers.

SECTION A — Architecture Overview

LexAegis is made of several pieces. Here's each one in plain English and how they connect.

Piece	What it does	Where it runs
Frontend	The website you see — login, upload documents, chat. Built with Next.js.	Vercel
Backend	The brain — receives questions, searches your documents, calls the LLM, returns grounded answers with citations. Built with FastAPI (Python).	Railway
Chroma	The vector database — stores the numeric "embeddings" of your document chunks so the backend can find relevant passages.	Railway (with a persistent volume)
Supabase	Handles user accounts and login (issues a secure token the backend trusts). A managed service — never containerized.	Supabase cloud
Gemini	Google's hosted LLM. Does the actual reasoning/answer generation.	Google cloud
Railway	The cloud host that runs your backend and Chroma containers.	—
Vercel	The cloud host that builds and serves your frontend .	—

How they connect



- 1. You log in → Supabase gives the browser a **JWT** (a signed token proving who you are).
- 2. The browser loads the frontend from Vercel.

- 3. Every action (upload, chat) is an API call to the **backend on Railway**, carrying the JWT.
- 4. The backend **verifies** the JWT using the Supabase JWT secret.
- 5. For a question, the backend searches **Chroma** for relevant document chunks.
- 6. It sends those chunks + your question to **Gemini**, which writes a grounded answer with [S1] citations. The backend returns it to the frontend.

For deeper diagrams see [DEPLOYMENT_ARCHITECTURE.md](#).

SECTION B — Required Accounts

Create these five free accounts (in this order):

#	Account	Sign up at	Used for
1	GitHub	https://github.com/signup	Stores your code; Railway & Vercel deploy from it
2	Supabase	https://supabase.com	User login / auth
3	Google AI Studio (Gemini)	https://aistudio.google.com	The LLM API key
4	Railway	https://railway.app	Hosts backend + Chroma
5	Vercel	https://vercel.com	Hosts the frontend

Sign in to Railway and Vercel **with your GitHub account** — it makes connecting your repo one click.

Step 0 — Put your code on GitHub

If the project isn't on GitHub yet:

```
# from the repo root
git init                # if not already a repo
git add .
git commit -m "LexAegis AI"
# create an empty repo on github.com, then:
git remote add origin https://github.com/<you>/lexaegis-ai.git
git branch -M main
git push -u origin main
```

Where to obtain each secret

Supabase — create a project first (Dashboard → **New project**; pick a region near you; save the database password). Then:

Value	Where to find it
Supabase URL	Dashboard → Project Settings → Data API → <i>Project URL</i> (looks like https://abcd1234.supabase.co)
Supabase anon key	Project Settings → API Keys → <i>anon</i> / <i>public</i> key
Supabase service role key	Project Settings → API Keys → <i>service_role</i> key (⚠ secret — server only)
Supabase JWT secret	Project Settings → API Keys → JWT Keys (or <i>Data API</i> → <i>JWT Settings</i>) → <i>JWT Secret</i>

Supabase's dashboard wording changes occasionally. The four things you need are always: the **Project URL**, the **anon** key, the **service_role** key, and the **JWT secret**. The anon key is safe to expose in the browser; the service_role key and JWT secret are **server-only**.

Gemini API key — go to <https://aistudio.google.com/apikey> → **Create API key** → copy it. This single key is your **GEMINI_API_KEY**.

SECTION C — Production Environment Variables

Below is every variable, where it lives, whether it's required, where to get it, and an example. **Public** (`NEXT_PUBLIC_*`) values are safe in the browser; everything else is a server secret — never put a secret in a `NEXT_PUBLIC_*` variable.

Frontend (set in **Vercel** → Project → Environment Variables)

Variable	Required	Where to get it	Example
NEXT_PUBLIC_API_BASE	✓	Your Railway backend URL + <code>/api/v1</code> (Section D step 6)	<code>https://lexaegis-backend.up.railway.app/api/v1</code>
NEXT_PUBLIC_SUPABASE_URL	✓	Supabase → Project URL	<code>https://abcd1234.supabase.co</code>
NEXT_PUBLIC_SUPABASE_ANON_KEY	✓	Supabase → anon key	<code>eyJhbGciOi...</code> (long)

Backend (set in **Railway** → backend service → Variables)

Variable	Required	Where to get it	Example
LLM_PROVIDER	✓	fixed value	<code>gemini</code>
GEMINI_API_KEY	✓	Google AI Studio	<code>AIza...</code>
GEMINI_MODEL	 (default flash)	choose	<code>gemini-2.5-flash</code>
ENVIRONMENT	✓	fixed value	<code>production</code>
CORS_ORIGINS	✓	your Vercel URL	<code>https://lexaegis.vercel.app</code>
SUPABASE_URL	✓	Supabase → Project URL	<code>https://abcd1234.supabase.co</code>
SUPABASE_ANON_KEY	✓	Supabase → anon key	<code>eyJhbGciOi...</code>
SUPABASE_SERVICE_ROLE_KEY	✓	Supabase → service_role key	<code>eyJhbGciOi...</code>
SUPABASE_JWT_SECRET	✓	Supabase → JWT secret	<code>super-secret-string</code>
LOG_JSON		fixed value	<code>true</code>

Chroma (set in **Railway** → Chroma service → Variables)

Variable	Required	Where to get it	Example
IS_PERSISTENT	✓	fixed value	<code>TRUE</code>
ANONYMIZED_TELEMETRY		fixed value	<code>FALSE</code>

...and on the **backend** service, point it at Chroma:

Variable	Required	Value
RETRIEVAL_VECTOR_STORE	✓	<code>chroma</code>
CHROMA_USE_HTTP_CLIENT	✓	<code>true</code>
CHROMA_HOST	✓	the Chroma service's private hostname (Railway shows it)
CHROMA_PORT	✓	<code>8000</code>

Supabase (managed — no env vars to host; you only *consume* its keys above)

Item	Required	Where
Project URL, anon key, service_role key, JWT secret	✓	Supabase dashboard (Section B)

Gemini (managed — one key, consumed by the backend)

Item	Required	Where
GEMINI_API_KEY	✓	Google AI Studio (Section B)

A ready-to-edit copy of all backend values lives in [deployment/production.env.example](#).

SECTION D — Railway Deployment (backend + Chroma)

Railway will host two services in one project: the **backend** and **Chroma**.

1. Create a project

1. Go to <https://railway.app> and **Login with GitHub**.
2. Click **New Project**.

2. Connect your GitHub repo

1. Choose **Deploy from GitHub repo**.
2. Authorize Railway to access your repos if prompted, then pick `lexaegis-ai`.
3. Railway creates a service from the repo. We'll configure it next.

3. Create the backend service

1. Open the service Railway just created → **Settings**.
2. **Source / Root Directory**: set to `backend`.
 - Railway auto-detects `backend/Dockerfile` and builds with it (CPU-only — no GPU packages; see [LLM_PROVIDER_GUIDE.md](#)).
3. **Settings** → **Networking** → **Generate Domain** — this gives a public URL like `https://lexaegis-backend.up.railway.app`. **Write it down** — it's your `NEXT_PUBLIC_API_BASE` (with `/api/v1` appended).

4. Add the Chroma service + persistent volume

1. In the same project, click **New** → **Database / Docker Image** → **Deploy a Docker Image** → enter `chromadb/chroma:1.0.0`.
2. Open the Chroma service → **Variables** → add:
 - `IS_PERSISTENT=TRUE`
 - `ANONYMIZED_TELEMETRY=FALSE`
3. **Volume (persistence — critical)**: Chroma service → **Settings** → **Volumes** → **Add Volume** → mount path `/data`. This is where embeddings are stored; it **survives restarts and redeloys**. Without it you lose all uploaded data on every deploy.
4. Note the Chroma service's **private network hostname** (Railway → Chroma → **Settings** → **Networking** → **Private Networking**, e.g. `chroma.railway.internal`).

5. Add environment variables (backend)

Open the **backend** service → **Variables** → **Raw Editor**, and paste (filling in real values from Sections B/C):

```
ENVIRONMENT=production
LOG_JSON=true
LLM_PROVIDER=gemini
GEMINI_API_KEY=AIza...your-key...
GEMINI_MODEL=gemini-2.5-flash

CORS_ORIGINS=https://your-frontend.vercel.app

SUPABASE_URL=https://abcd1234.supabase.co
SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...
SUPABASE_JWT_SECRET=your-jwt-secret

RETRIEVAL_VECTOR_STORE=chroma
CHROMA_USE_HTTP_CLIENT=true
```

```
CHROMA_HOST=chroma.railway.internal
CHROMA_PORT=8000
```

You don't have the Vercel URL yet — put a placeholder in `CORS_ORIGINS` now and come back to fix it after Section E. Railway redeploys automatically when you change a variable.

6. Verify deployment

1. Backend service → **Deployments** tab → watch the build log. CPU-only torch keeps it reasonably fast. Wait for **"Success / Active"**.
2. Open the **Logs** and look for:
 - `LLM_PROVIDER = gemini`
 - `[LLM HEALTH] Gemini reachable ... model=gemini-2.5-flash available=True`
 - If you see `GEMINI_API_KEY is empty` → your key variable is missing/typo'd.

7. Verify health endpoints

In a terminal (replace with your Railway domain):

```
curl https://lexaegis-backend.up.railway.app/api/v1/health # {"status":"ok",...}
curl https://lexaegis-backend.up.railway.app/api/v1/ready
```

Both should return JSON, not an error page.

SECTION E — Vercel Deployment (frontend)

1. Import the repository

1. Go to <https://vercel.com> → **Login with GitHub**.
2. **Add New...** → **Project** → **Import** your `lexaegis-ai` repo.

2. Configure build settings

1. **Root Directory:** click **Edit** → set to `frontend`.
2. **Framework Preset:** Next.js (auto-detected — leave defaults).
3. Build command `next build` and output are auto-configured. Do **not** override.

3. Configure environment variables

Still on the import screen (or later: Project → **Settings** → **Environment Variables**), add the three frontend variables:

```
NEXT_PUBLIC_API_BASE      = https://lexaegis-backend.up.railway.app/api/v1
NEXT_PUBLIC_SUPABASE_URL  = https://abcd1234.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY = eyJ...
```

These are **baked in at build time**. If you change one later, you must **Redeploy** (Deployments → ... → Redeploy) for it to take effect.

4. Deploy

Click **Deploy**. Vercel builds the Next.js app (no Docker needed) and gives you a URL like `https://lexaegis.vercel.app`.

5. Verify frontend

1. Open the Vercel URL — the login page should render.
2. **Go back to Railway** (Section D step 5) and set `CORS_ORIGINS=https://lexaegis.vercel.app` (your real Vercel URL), exactly, no trailing slash. Save → backend redeploys.
 - Without this, the browser blocks API calls with a CORS error.

SECTION F — Production Validation Checklist

Do these in order in your browser at the Vercel URL. Exact API tests are in [DEPLOYMENT_VALIDATION.md](#) if you prefer curl.

Test	How	Expected
Login	Sign up / log in on the site	You reach the chat page; no console errors
Upload	Upload a PDF/DOCX on the documents page	Success message + the doc appears in the list
Retrieval	Ask a question answerable from that doc	An answer appears (not "insufficient evidence")
Chat	Ask a follow-up	Coherent answer; backend logs show a <code>[GEMINI]</code> line
Citations	Look at the answer	Inline <code>[S1]/[S2]</code> tags + a sources list
Groundedness	Ask something NOT in your docs (e.g. "capital of Mars?")	It refuses / says insufficient evidence rather than inventing
Evaluation page	Open the Evaluation Dashboard	Shows metrics if a report exists — see note below

Quick CORS/health check from a terminal:

```
curl https://<railway-backend>/api/v1/health
# In the browser devtools Network tab, API calls should be 200, not CORS errors.
```

Evaluation page in production: the dashboard reads a static report (`evaluation/results/latest.json`) produced by the **offline** eval harness. On Railway the backend image is built from `./backend` only, so that file isn't present and the page will say "No evaluation report yet" — this is expected, not a bug. To populate it in production, run the harness locally (`python evaluation/evaluate_local.py`), commit the generated `evaluation/results/latest.json`, and copy it into the backend image (add a `COPY` step), **or** treat the dashboard as a local/dev-only feature. Locally (Docker or native) it works out of the box.

SECTION G — Troubleshooting

Symptom	Cause	Fix
Frontend loads but every API call fails with CORS error	<code>CORS_ORIGINS</code> on the backend doesn't match the Vercel URL	Set <code>CORS_ORIGINS</code> to the exact Vercel URL (no trailing slash); redeploy backend
Frontend can't reach backend at all	<code>NEXT_PUBLIC_API_BASE</code> wrong or missing <code>/api/v1</code>	Fix it in Vercel → Redeploy (it's build-time)
Login does nothing / "Supabase not configured"	<code>NEXT_PUBLIC_SUPABASE_URL</code> / <code>ANON_KEY</code> blank or not rebuilt	Set both in Vercel, Redeploy
Compose warns <code>NEXT_PUBLIC_SUPABASE_URL not set</code> (local Docker)	Compose doesn't read <code>frontend/.env.local</code>	<code>cp .env.docker.example .env</code> and fill it (see DOCKER_FOR_BEGINNERS.md)
Backend logs <code>GEMINI_API_KEY is empty</code>	Key variable missing/typo on Railway	Add <code>GEMINI_API_KEY</code> to the backend service variables
Backend logs <code>Gemini API UNREACHABLE</code>	Bad key, wrong model name, or no network	Test the key: <code>curl "https://generativelanguage.googleapis.com/v1beta/models?key=YOUR_KEY"</code>
Answers are always generic/"insufficient evidence"	No documents uploaded, or Chroma not connected	Upload a doc; check <code>CHROMA_HOST/PORT</code> + that the Chroma volume is mounted at <code>/data</code>

Symptom	Cause	Fix
Uploaded documents disappear after a redeploy	Chroma has no persistent volume	Add a Railway volume mounted at <code>/data</code> on the Chroma service
401 on every request	JWT not verified	Ensure <code>SUPABASE_JWT_SECRET</code> on the backend matches the Supabase project
Railway build fails downloading GPU <code>torch/nvidia_*</code>	Old image without the CPU-torch fix	Pull the latest code; <code>backend/Dockerfile</code> installs CPU-only torch
Chroma container marked unhealthy	Old healthcheck used <code>curl</code> (not in the image)	Latest compose uses a bash <code>/dev/tcp</code> check — pull latest code
Evaluation page empty in production	Report file not in the backend image	Expected — see the note in Section F

You're done 🎉

- Frontend: `https://<your-app>.vercel.app`
- Backend: `https://<your-backend>.up.railway.app/api/v1`
- Data persists in the Railway Chroma volume.
- LLM served by Gemini; auth by Supabase.

To run the same stack on your own computer instead, see [DOCKER_FOR_BEGINNERS.md](#). To understand provider switching (Ollama ↔ Gemini), see [LLM_PROVIDER_GUIDE.md](#).